LAB 1:

Foundations

พื้นฐานระบบแนะนำอิงเนื้อหา

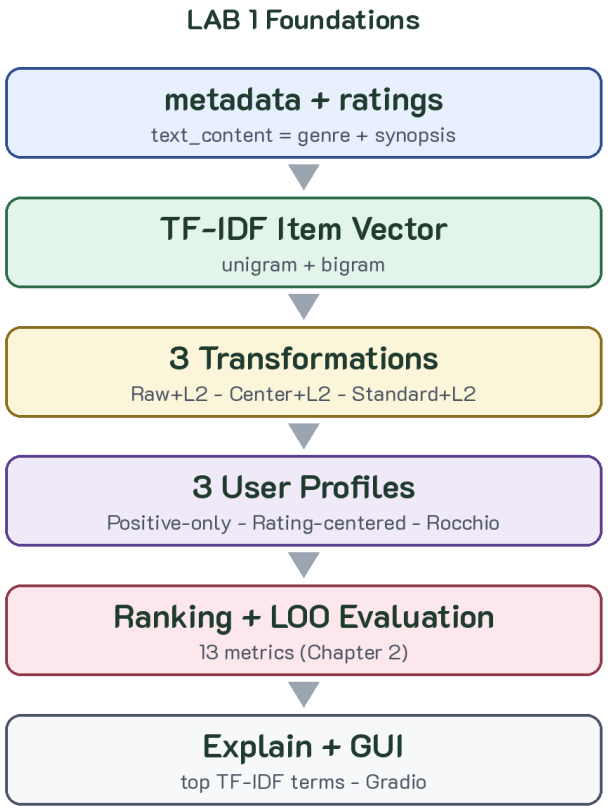
204729

Recommender Systems

ผู้ช่วยศาสตราจารย์ ดร.ดุษฎี ประเสริฐธิติพงษ์

Text Vector - 3 Profiles

Evaluation - GUI



อธิบายแนวคิดของ Source Code ทีละ Part

TF-IDF - Transformation - Profiles - LOO Metrics - Gradio

COMPUTER SCIENCE

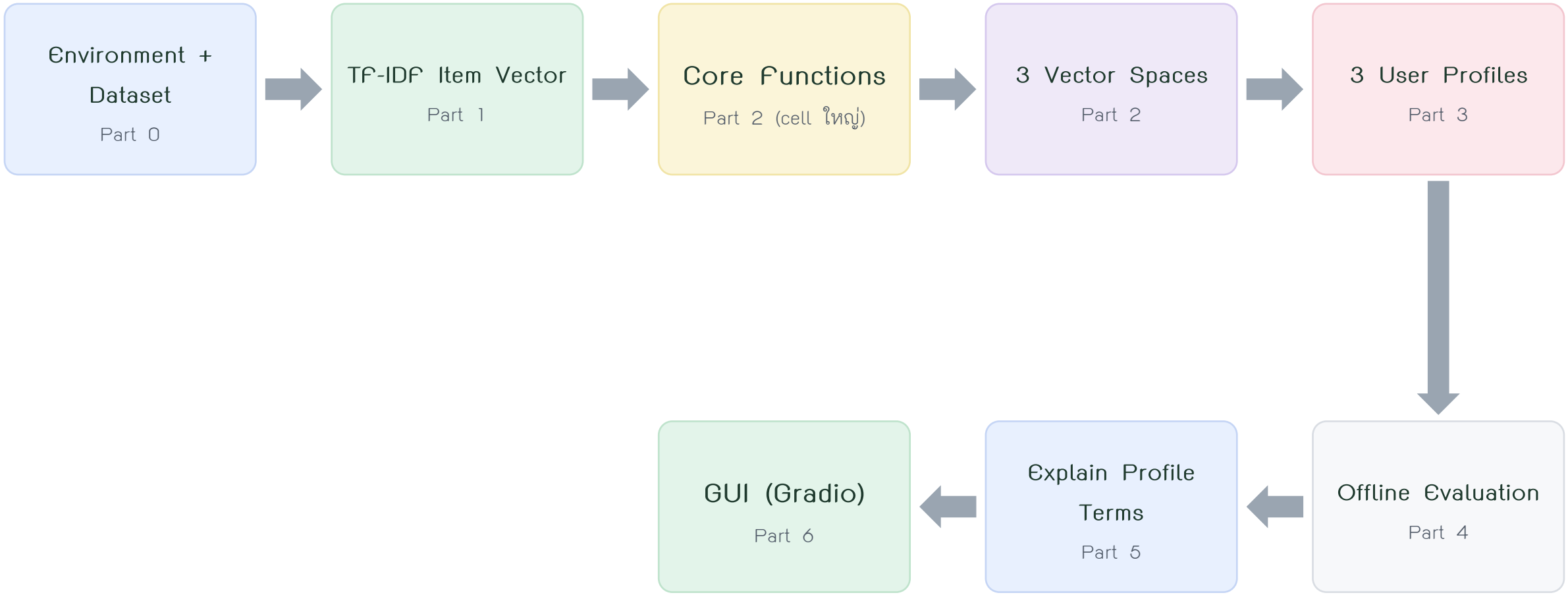
LAB 1 คืออะไร

ภาพรวม

- Notebook **Text-only** ตามบทที่ 3: สร้าง Item Vector จาก `genre + synopsis` ด้วย TF-IDF แล้วเดินผ่าน **Vector Transformation** ⟶ **User Profile** ⟶ **Cosine Ranking** ⟶ **Evaluation** ⟶ **GUI**
- จุดเน้นคือ **User Profile 3 วิธี**: Positive-only (เฉลี่ยหนังที่ชอบ), Rating-centered (น้ำหนัก rating − ค่าเฉลี่ยของผู้ใช้เอง), Rocchio (α·mean(liked) − β·mean(disliked))
- ประเมินด้วย **Leave-One-Out + Full Catalog** เดียวกันทุก configuration และ metric จากบทที่ 2 ครบสองกลุ่ม: Accuracy/Ranking (Precision, Recall, F1, HR, MRR, MAP, nDCG, AUC) และ Beyond-Accuracy (Coverage, Gini, ILD, Novelty, Calibration)
- **Core Functions** (transform, profiles, rank, metrics, folds, `evaluate_config`) เขียนไว้ใน cell เดียว และ **ใช้ซ้ำแบบเดียวกันใน LAB 2 และ LAB 3** — เข้าใจ cell นี้ให้ดีเท่ากับเข้าใจสามเล่ม
- ไม่มีแบบฝึกหัดในเล่มนี้ แต่ใน code มี comment ชี้ประเด็นให้สังเกต ; GUI แสดง User History, Top Profile Terms และ Recommended Movies

Dataset: MyDrive/Colab Notebooks/Dataset_Chapter3_4 (metadata.csv + ratings.csv)

โครงสร้าง Notebook LAB 1 (Part 0 → Part 6)



Part 2 cell 'Core Functions' นิยามทุกฟังก์ชันที่ Part 3–6 เรียกใช้

แผนที่ของ Source Code LAB 1

Part	หน้าที่	ฟังก์ชัน / ตัวแปรสำคัญ	ผลลัพธ์
0	ติดตั้ง, mount Drive, โหลด + rename schema, สร้าง `text_content`	`DATA_DIR`, `items`, `ratings`, `ITEM_INDEX`, `N_USERS`, `POPULARITY`	ข้อมูลพร้อมใช้ทั้ง 3 LAB
1	TF-IDF จาก `text_content`	`tfidf`, `X_tfidf_raw`	เมทริกซ์ 12 × |vocab|
2	Core Functions	`transform_vectors`, `profile_*` ×3, `get_profile`, `rank_user`, metric ×13, `loo_folds`, `evaluate_config`	เครื่องมือกลาง
2	สร้าง 3 vector space	`VECTOR_SPACES`	Raw+L2 / Center+L2 / Standard+L2
3	เทียบ 3 profile ของผู้ใช้ตัวอย่าง	`profiles`, ตาราง Top-3, เมทริกซ์ cosine `M`	โปรไฟล์ต่างกันแค่ไหน
4	LOO evaluation ทุก transformation × profile	`folds`, `lab1_results`	ตาราง 9 แถว × 15 metric
5	อธิบายโปรไฟล์ด้วยคำ TF-IDF	`top_profile_terms`	คำที่มีน้ำหนักสูงสุดต่อ profile
6	GUI	`gui_recommend`, `gr.Interface`	History + Terms + Recommendations

1

Environment, Dataset และ TF-IDF

Part 0 – Part 1: เตรียมข้อมูลให้เป็น canonical schema แล้วสร้าง Item Vector

- `DATA_DIR` เลือกอัตโนมัติระหว่าง `Dataset_Chapter3_4/data` กับ `Dataset_Chapter3_4`
- `text_content = genre + synopsis` เป็นข้อความเดียวที่ทุก representation ใช้

Part 0: Environment และตำแหน่ง Dataset

```
!pip -q install gradio gensim sentence-transformers transformers

from google.colab import drive
drive.mount("/content/drive")

from pathlib import Path
import math, random, warnings
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import normalize, StandardScaler
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import roc_auc_score, mean_absolute_error,
        mean_squared_error
warnings.filterwarnings("ignore")

BASE = Path("/content/drive/MyDrive/Colab Notebooks")
DATA_ROOT = BASE / "Dataset_Chapter3_4"
if (DATA_ROOT / "data").exists():
    DATA_DIR = DATA_ROOT / "data"
else:
    DATA_DIR = DATA_ROOT
```

อธิบายโค้ด

เตรียม environment ครั้งเดียวสำหรับสามเล่ม

- ติดตั้ง `gensim`, `sentence-transformers`, `transformers` ไว้ล่วงหน้าแม้ LAB 1 ใช้แค่ TF-IDF — เพื่อให้ Part 0 **เหมือนกันทุกเล่ม**
- `DATA_DIR` รองรับสองโครงสร้างโฟลเดอร์ (มี/ไม่มีโฟลเดอร์ย่อย `data`) ⟶ ลดปัญหา path ผิดในห้องเรียน
- `normalize` + `StandardScaler` ใช้ใน transformation ; `cosine_similarity` ใช้จัดอันดับ ; `roc_auc_score` ใช้คำนวณ AUC
- `warnings.filterwarnings("ignore")` ซ่อน warning ของ sklearn/transformers ให้ output อ่านง่าย

Part 0: โหลด + rename schema + ตัวแปรกลาง (1/2)

```
items = pd.read_csv(DATA_DIR / "metadata.csv")
ratings = pd.read_csv(DATA_DIR / "ratings.csv")

rename_items = {"movie_id": "item_id", "id": "item_id", "movie_title":
        "title", "name": "title",
                "genres": "genre", "description": "synopsis", "plot":
                        "synopsis", "summary": "synopsis"}
for old, new in rename_items.items():
    if old in items.columns and new not in items.columns:
        items = items.rename(columns={old: new})
rename_r = {"uid": "user_id", "user": "user_id", "iid": "item_id",
        "movie_id": "item_id", "score": "rating"}
for old, new in rename_r.items():
    if old in ratings.columns and new not in ratings.columns:
        ratings = ratings.rename(columns={old: new})

required_i = ["item_id", "title", "genre", "synopsis"]
required_r = ["user_id", "item_id", "rating"]
```

อธิบายโค้ด

สามตัวแปร global ที่ทุกฟังก์ชันอ้างถึง

- rename เฉพาะเมื่อยังไม่มีชื่อมาตรฐาน แล้ว `assert` คอลัมน์บังคับ → fail เร็วพร้อมแสดงคอลัมน์ที่พบ
- `item_id`, `user_id` เป็น `str` ทั้งสองตาราง → join และ `ITEM_INDEX` ตรงกัน ; `rating` เป็นตัวเลข
- `text_content` = genre + synopsis → ข้อความหนึ่งก้อนต่อหนัง ให้ทุก item อยู่ใน vocabulary เดียวกัน
- `ITEM_INDEX` map item_id → แถวของเมทริกซ์ ; `N_USERS` และ `POPULARITY` (จำนวนผู้ใช้ที่ให้คะแนนแต่ละเรื่อง — นับทุก rating ไม่เฉพาะ positive) ใช้คำนวณ Novelty
- ค่าเหล่านี้คำนวณจาก `ratings` ทั้งหมดครั้งเดียว ไม่ได้คำนวณใหม่ต่อ fold

Part 0: โหลด + rename schema + ตัวแปรกลาง (2/2)

```python
assert all(c in items.columns for c in required_i),
       items.columns.tolist()
assert all(c in ratings.columns for c in required_r),
       ratings.columns.tolist()

items["item_id"] = items["item_id"].astype(str)
ratings["item_id"] = ratings["item_id"].astype(str)
ratings["user_id"] = ratings["user_id"].astype(str)
ratings["rating"] = pd.to_numeric(ratings["rating"])
items["genre"] = items["genre"].fillna("").astype(str)
items["synopsis"] = items["synopsis"].fillna("").astype(str)
items["text_content"] = (items["genre"] + " " +
       items["synopsis"]).str.strip()

ITEM_INDEX = {iid: i for i, iid in enumerate(items["item_id"])}
N_USERS = max(1, ratings["user_id"].nunique())
POPULARITY = ratings.groupby("item_id")["user_id"].nunique().to_dict()
```

อธิบายโค้ด

สามตัวแปร global ที่ทุกฟังก์ชันอ้างถึง

- rename เฉพาะเมื่อยังไม่มีชื่อมาตรฐาน แล้ว `assert` คอลัมน์บังคับ ⟶ fail เร็วพร้อมแสดงคอลัมน์ที่พบ
- `item_id`, `user_id` เป็น `str` ทั้งสองตาราง ⟶ join และ `ITEM_INDEX` ตรงกัน ; `rating` เป็นตัวเลข
- `text_content` = genre + synopsis ⟶ ข้อความหนึ่งก้อนต่อหนัง ให้ทุก item อยู่ใน vocabulary เดียวกัน
- `ITEM_INDEX` map item_id ⟶ แถวของเมทริกซ์ ; `N_USERS` และ `POPULARITY` (จำนวนผู้ใช้ที่ให้คะแนนแต่ละเรื่อง — นับทุก rating ไม่เฉพาะ positive) ใช้คำนวณ Novelty
- ค่าเหล่านี้คำนวณจาก `ratings` ทั้งหมดครั้งเดียว ไม่ได้คำนวณใหม่ต่อ fold

Part 1: TF-IDF Item Vector

```
# ใช้ genre + synopsis เพื่อให้ทุก item อยู่ใน vocabulary เดียวกัน
tfidf = TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)
X_tfidf_raw = tfidf.fit_transform(items["text_content"]).toarray()
print("Vocabulary =", len(tfidf.get_feature_names_out()), "Shape =",
        X_tfidf_raw.shape)
```

อธิบายโค้ด

Item Vector ที่ตีความได้

- แต่ละแถว = หนัง 1 เรื่อง, แต่ละคอลัมน์ = คำหรือคู่คำ (`ngram_range=(1,2)`) ⟶ มิติมีชื่อ ทำให้ Part 5 อธิบายโปรไฟล์เป็นคำได้
- TF สูงเมื่อคำปรากฏบ่อยในเรื่องนั้น, IDF สูงเมื่อคำหายากใน catalog ⟶ ชื่อแนวหนังที่พบทุกเรื่องถูกกดน้ำหนัก
- `min_df=1` เพราะมีเพียง 12 เอกสาร ; `.toarray()` ให้ dense matrix เพื่อทำ centering/standardize ได้
- ตัวแปรชื่อ `_raw` เพราะ transformation จะทำใน Part 2 ผ่าน `transform_vectors` (sklearn L2 ให้อยู่แล้ว แต่จะถูก normalize ซ้ำซึ่งไม่เปลี่ยนค่า)

2

Core Functions (1): Transformation และ 3 User Profiles

Part 2 cell ใหญ่ — ส่วนที่ 1

- `transform_vectors(X, mode)` 3 โหมด
- `_unit`, `profile_positive_only`, `profile_rating_centered`, `profile_rocchio`, `PROFILE_METHODS`, `get_profile`

Part 2: transform_vectors และ _unit

```python
LIKED_THRESHOLD = 4.0
DISLIKED_THRESHOLD = 2.0

def transform_vectors(X, mode="Raw+L2"):
    X = np.asarray(X, dtype=float)
    if mode == "Raw+L2":
        return normalize(X)
    if mode == "Center+L2":
        return normalize(X - X.mean(axis=0, keepdims=True))
    if mode == "Standard+L2":
        return normalize(StandardScaler().fit_transform(X))
    raise ValueError(mode)

def _unit(v):
    v = np.asarray(v, dtype=float)
    n = np.linalg.norm(v)
    return v if n == 0 else v / n
```

อธิบายโค้ด

geometry ของ feature space

- Raw+L2: ตัดผลของขนาดเวกเตอร์ ⟶ cosine วัดทิศทางล้วน ; กับ TF-IDF แทบไม่เปลี่ยน
- Center+L2: ลบค่าเฉลี่ยของแต่ละมิติทั่ว catalog ⟶ คำที่ทุกเรื่องมีร่วมกันหายไป, เกิดค่าลบ, cosine อาจติดลบ ⟶ ความคล้ายกระจายกว้างขึ้น
- Standard+L2: ทุกมิติ mean 0, std 1 ⟶ มิติหายาก (คำที่พบเรื่องเดียว) ถูกขยายเท่ามิติสำคัญ — เสี่ยง noise กับ 12 เรื่อง
- **เหตุผลที่ต้องเทียบ:** transformation เปลี่ยน geometry จึงเปลี่ยนทั้ง User Profile (ค่าเฉลี่ยของเวกเตอร์) และ ranking
- `_unit` = L2-normalize ที่ทนต่อเวกเตอร์ศูนย์ ใช้ปิดท้ายทุก profile
- `LIKED_THRESHOLD=4`, `DISLIKED_THRESHOLD=2` เป็นเกณฑ์ชอบ/ไม่ชอบที่ profile ทุกแบบใช้ร่วมกัน

Part 2: profile_positive_only และ profile_rating_centered

```python
def profile_positive_only(user_id, X, ratings_df):
    r = ratings_df[ratings_df["user_id"].astype(str) == str(user_id)]
    liked = r[r["rating"] >= LIKED_THRESHOLD]
    idx = [ITEM_INDEX[str(i)] for i in liked["item_id"] if str(i) in
           ITEM_INDEX]
    if not idx:                                    # ไม่มีหนังที่ชอบ ->
           ใช้ทุกเรื่องที่เคยดู
        idx = [ITEM_INDEX[str(i)] for i in r["item_id"] if str(i) in
               ITEM_INDEX]
    return _unit(X[idx].mean(axis=0)) if idx else np.zeros(X.shape[1])

def profile_rating_centered(user_id, X, ratings_df):
    r = ratings_df[ratings_df["user_id"].astype(str) ==
           str(user_id)].copy()
    r = r[r["item_id"].astype(str).isin(ITEM_INDEX)]
    if r.empty: return np.zeros(X.shape[1])
    idx = np.array([ITEM_INDEX[str(i)] for i in r["item_id"]])
    w = r["rating"].astype(float).values - float(r["rating"].mean())
           # rating - user mean
    denom = np.abs(w).sum()
    p = X[idx].mean(axis=0) if denom <= 1e-12 else (X[idx] * w[:,
           None]).sum(axis=0) / denom
    return _unit(p)
```

อธิบายโค้ด

สองสูตรแรก

- Positive-only = ค่าเฉลี่ยธรรมดาของเวกเตอร์หนังที่ rating ≥ 4 ; ถ้าผู้ใช้ไม่มีหนังที่ชอบเลย fallback ไปเฉลี่ยทุกเรื่อง (ยังได้โปรไฟล์ ไม่ใช่ศูนย์)
- Rating-centered ถ่วงน้ำหนักด้วย `rating − mean_rating_of_user` ⟶ **จุดกลางเป็นของผู้ใช้แต่ละคน** ไม่ใช่ค่าคงที่ 3: ผู้ใช้ใจดีที่ให้ 4 ทุกเรื่อง จะได้น้ำหนัก 0 ทั้งหมด ⟶ fallback เป็นค่าเฉลี่ย
- หารด้วย `Σ|w|` เพราะน้ำหนักมีทั้งบวกและลบ ; หนังที่ต่ำกว่าค่าเฉลี่ยของตนเอง **ผลัก** โปรไฟล์ออก
- ทั้งสองรับ `ratings_df` เป็นพารามิเตอร์ ⟶ ตอนประเมินส่ง `train` (ไม่มี held-out) ได้
- ต่างจาก Lab ชุดก่อนที่ใช้ `rating − 3`: centering ด้วยค่าเฉลี่ยของผู้ใช้เองแก้ user bias (คนให้คะแนนเข้มงวด/ใจดี)

Part 2: profile_rocchio, PROFILE_METHODS และ get_profile

```python
def profile_rocchio(user_id, X, ratings_df, alpha=1.0, beta=1.0):
    r = ratings_df[ratings_df["user_id"].astype(str) == str(user_id)]
    liked = r[r["rating"] >= LIKED_THRESHOLD]
    disliked = r[r["rating"] <= DISLIKED_THRESHOLD]
    li = [ITEM_INDEX[str(i)] for i in liked["item_id"] if str(i) in
          ITEM_INDEX]
    di = [ITEM_INDEX[str(i)] for i in disliked["item_id"] if str(i) in
          ITEM_INDEX]
    pos = X[li].mean(axis=0) if li else np.zeros(X.shape[1])
    neg = X[di].mean(axis=0) if di else np.zeros(X.shape[1])
    p = alpha * pos - beta * neg
    if np.linalg.norm(p) == 0:                    # ไม่มีทั้ง liked
          และ disliked
        hi = [ITEM_INDEX[str(i)] for i in r["item_id"] if str(i) in
              ITEM_INDEX]
        p = X[hi].mean(axis=0) if hi else p
    return _unit(p)

PROFILE_METHODS = {
    "Positive-only":   profile_positive_only,
    "Rating-centered": profile_rating_centered,
    "Rocchio":         profile_rocchio,
}

def get_profile(user_id, X, ratings_df, method, alpha=1.0, beta=1.0):
    if method == "Rocchio":
        return profile_rocchio(user_id, X, ratings_df, alpha, beta)
    return PROFILE_METHODS[method](user_id, X, ratings_df)
```

อธิบายโค้ด

Rocchio: ชอบดึงเข้า ไม่ชอบผลักออก

- โปรไฟล์ = α·mean(liked) – β·mean(disliked) ; หนังที่ rating 3 (กลาง ๆ) ไม่ถูกใช้เลย
- α, β ปรับได้จาก GUI (slider 0–2): β = 0 ⟶ เท่ากับ Positive-only ; β สูง ⟶ หลีกเลี่ยงลักษณะของหนังที่ไม่ชอบมากขึ้น
- ต่างจาก Rating-centered ตรงที่ **ไม่ถ่วงน้ำหนักตามระดับ rating** (5 กับ 4 เท่ากัน) แต่แยกกลุ่มชอบ/ไม่ชอบชัดเจน
- `PROFILE_METHODS` = dict ชื่อ ⟶ ฟังก์ชัน ; `get_profile` เป็นทางเข้าเดียวที่รับ `method` และส่ง α, β ให้ Rocchio ⟶ `rank_user`, `evaluate_config`, GUI เรียกผ่านจุดนี้
- fallback เมื่อไม่มีทั้ง liked/disliked ⟶ เฉลี่ยประวัติทั้งหมด เพื่อไม่ให้ cosine เป็น 0 ทุกเรื่อง

เปรียบเทียบ 3 User Profile

Profile	สูตร	ใช้หนังเรื่องใด	จุดเด่น	ข้อจำกัด
Positive-only	mean(v_i : $r_i \geq 4$)	เฉพาะที่ชอบ	ง่าย เสถียร ไม่ไวต่อ noise	ทิ้งข้อมูลจากหนังที่ไม่ชอบ ; 4 กับ 5 เท่ากัน
Rating-centered	$\sum(r_i - \bar{r}_u) \cdot v_i \,/\, \sum\lvert r_i - \bar{r}_u \rvert$	ทุกเรื่อง	แก้ user bias ; ใช้ negative feedback	ผู้ใช้ที่ให้คะแนนเท่ากันหมด ⟶ น้ำหนัก 0 ⟶ fallback
Rocchio	$\alpha \cdot$mean(liked) − $\beta \cdot$mean(disliked)	ชอบ (≥4) และไม่ชอบ (≤2)	ควบคุมได้ด้วย α, β ; ตีความง่าย	ไม่ใช้ rating 3 ; ไม่แยกระดับความชอบ

ทุกสูตรปิดท้ายด้วย _unit ⟶ เทียบกันได้เฉพาะทิศทาง

3

Core Functions (2): Ranking, Metrics และ evaluate_config

Part 2 cell ใหญ่ — ส่วนที่ 2

- `rank_user` คืน `(ranked, score, profile)`
- metric 8 ตัว (Accuracy/Ranking) + 5 ตัว (Beyond-Accuracy)
- `loo_folds` และ `evaluate_config` ที่รองรับ Full Catalog / Sampled Negatives

Part 2: rank_user

```python
def rank_user(user_id, X, ratings_df, method, k=None, candidates=None,
        alpha=1.0, beta=1.0):
    p = get_profile(user_id, X, ratings_df, method, alpha, beta)
    score = cosine_similarity(p.reshape(1, -1), X).ravel()
    seen = set(ratings_df.loc[ratings_df["user_id"].astype(str) ==
            str(user_id), "item_id"].astype(str))
    if candidates is None:
        candidates = [iid for iid in items["item_id"] if iid not in
                seen]
    ranked = sorted(candidates, key=lambda iid: score[ITEM_INDEX[iid]],
            reverse=True)
    return ranked[:k] if k else ranked, score, p
```

อธิบายโค้ด

ฟังก์ชันจัดอันดับเดียวสำหรับทุกกรณี

- สร้างโปรไฟล์ตาม `method` ⟶ cosine กับ **ทุก** item ⟶ เรียงเฉพาะ `candidates`
- `candidates=None` ⟶ ทุกเรื่องที่ยังไม่เห็นใน `ratings_df` (GUI ใช้แบบนี้) ; `evaluate_config` ส่ง candidate ที่เตรียมเองเข้ามา (Full Catalog หรือ Sampled Negatives)
- `k=None` คืนลำดับทั้งหมด (ต้องใช้กับ MRR/AUC) ; `k` ระบุ ⟶ ตัด Top-K
- คืน 3 อย่าง: ลำดับ, คะแนนทุก item (สำหรับ AUC), และโปรไฟล์ (GUI นำไปแสดงคำเด่น)
- `seen` มาจาก `ratings_df` ที่ส่งเข้ามา ⟶ ถ้าส่ง `train` held-out จะไม่อยู่ใน seen จึงเป็น candidate

Part 2: Accuracy / Ranking metrics (8 ตัว)

```python
def precision_at_k(recs, rel, k):
    return len(set(recs[:k]) & set(rel)) / k if k else 0.0
def recall_at_k(recs, rel, k):
    rel = set(rel); return len(set(recs[:k]) & rel) / len(rel) if rel
            else 0.0
def f1_at_k(recs, rel, k):
    p = precision_at_k(recs, rel, k); r = recall_at_k(recs, rel, k)
    return 0.0 if p + r == 0 else 2 * p * r / (p + r)
def hr_at_k(recs, rel, k):
    return float(bool(set(recs[:k]) & set(rel)))
def reciprocal_rank(recs, rel):
    rel = set(rel)
    for rank, iid in enumerate(recs, 1):
        if iid in rel: return 1.0 / rank
    return 0.0
def ap_at_k(recs, rel, k):
    rel = set(rel); hits = 0; s = 0.0
    for i, iid in enumerate(recs[:k], 1):
        if iid in rel:
            hits += 1; s += hits / i
    return s / min(len(rel), k) if rel else 0.0
def ndcg_at_k(recs, rel, k):
    rel = set(rel)
    dcg = sum(1 / math.log2(i + 2) for i, iid in enumerate(recs[:k]) if
            iid in rel)
    idcg = sum(1 / math.log2(i + 2) for i in range(min(len(rel), k)))
    return dcg / idcg if idcg else 0.0
```

อธิบายโค้ด

ถูกหรือไม่ vs ถูกที่ตำแหน่งใด

- **Set-based**: Precision@K = ถูก/K, Recall@K = ถูก/|rel|, F1 = ฮาร์มอนิก, HR@K = มีถูกอย่างน้อย 1 ; ใน LOO (|rel| = 1) Precision@3 ≤ 1/3 และ Recall@3 = HR@3
- **Rank-aware**: MRR = 1/อันดับแรกที่ถูก (บน ranked ทั้งหมด) ; MAP@K = เฉลี่ย precision ณ ตำแหน่งที่ถูก ; nDCG@K ให้ gain $1/\log_2(\text{rank}+1)$ — อันดับ 1 = 1.0, 2 ≈ 0.63, 3 = 0.5
- AUC คำนวณใน `evaluate_config` ด้วย `roc_auc_score` จาก score ของ candidate ทั้งหมด
- ทุกฟังก์ชันรับ `rel` เป็น iterable ⟶ ใช้ได้ทั้ง `[heldout]` (LAB 1–2) และ relevant หลายเรื่อง (LAB 3)

Part 2: Beyond-Accuracy metrics (5 ตัว) (1/2)

```python
def coverage(all_recs):
    return len(set(all_recs)) / len(items) if len(items) else 0.0

def gini_coefficient(all_recs):
    counts = pd.Series(all_recs).value_counts()
    x = np.array([counts.get(iid, 0) for iid in items["item_id"]],
            dtype=float)
    if x.sum() == 0: return 0.0
    x = np.sort(x); n = len(x)
    return float((2 * np.sum(np.arange(1, n + 1) * x) / (n * x.sum()))
            - (n + 1) / n)

def ild(item_ids, X):
    idx = [ITEM_INDEX[i] for i in item_ids if i in ITEM_INDEX]
    if len(idx) < 2: return 0.0
    D = 1 - cosine_similarity(X[idx])
    v = D[np.triu_indices_from(D, k=1)]
    return float(v.mean()) if len(v) else 0.0

def novelty(item_ids):
    vals = []
    for iid in item_ids:
        pop = max(1, POPULARITY.get(iid, 1))
```

อธิบายโค้ด

คุณภาพเชิงระบบและประสบการณ์

- Coverage = สัดส่วน catalog ที่เคยถูกแนะนำ (จาก `all_recs` รวมทุกผู้ใช้) ; Gini = ความกระจุกของความถี่ที่แต่ละเรื่องถูกแนะนำ (เรื่องที่ไม่เคยถูกแนะนำนับ 0)
- ILD = ค่าเฉลี่ย (1 − cosine) ของคู่ใน Top-K วัดใน space X ที่ส่งเข้ามา ⟶ เทียบข้าม transformation/representation ได้ไม่ตรง
- Novelty = $-\log_2$(popularity/N_USERS) ; popularity นับผู้ใช้ทุกคนที่ให้คะแนน (ไม่ใช่เฉพาะที่ชอบ)
- Calibration (เวอร์ชันนี้) = 1 − ½·$\sum$|p_hist − p_rec| คือ 1 − total variation distance ของการกระจายแนวหนังระหว่างประวัติกับ Top-K ⟶ 1 = สัดส่วนเหมือนกัน ; คืน `nan` ถ้าไม่มี genre
- genre คั่นด้วย `|` หรือ `,` ถูก split และนับซ้ำตามจำนวนเรื่อง

Part 2: Beyond-Accuracy metrics (5 ตัว) (2/2)

```
        vals.append(-math.log2(pop / N_USERS))
    return float(np.mean(vals)) if vals else 0.0

def calibration_genre(user_id, rec_ids, ratings_df):
    hist = ratings_df[ratings_df["user_id"].astype(str) == str(user_id)]
    hist_genres = []
    for iid in hist["item_id"].astype(str):
        if iid in ITEM_INDEX:
            hist_genres += [g.strip() for g in
                    str(items.iloc[ITEM_INDEX[iid]]["genre"]).replace("|
                            ", ",").split(",") if g.strip()]
    rec_genres = []
    for iid in rec_ids:
        rec_genres += [g.strip() for g in
                str(items.iloc[ITEM_INDEX[iid]]["genre"]).replace("|",
                        ",").split(",") if g.strip()]
    if not hist_genres or not rec_genres: return np.nan
    cats = sorted(set(hist_genres) | set(rec_genres))
    hp = np.array([hist_genres.count(c) for c in cats], float); hp /=
            hp.sum()
    rp = np.array([rec_genres.count(c) for c in cats], float); rp /=
            rp.sum()
    return float(1 - 0.5 * np.abs(hp - rp).sum())
```

อธิบายโค้ด

คุณภาพเชิงระบบและประสบการณ์

- Coverage = สัดส่วน catalog ที่เคยถูกแนะนำ (จาก `all_recs` รวมทุกผู้ใช้) ; Gini = ความกระจุกของความถี่ที่แต่ละเรื่องถูกแนะนำ (เรื่องที่ไม่เคยถูกแนะนำนับ 0)
- ILD = ค่าเฉลี่ย (1 − cosine) ของคู่ใน Top-K วัดใน space `X` ที่ส่งเข้ามา ⟶ เทียบข้าม transformation/representation ได้ไม่ตรง
- Novelty = $-\log_2$(popularity/N_USERS) ; popularity นับผู้ใช้ทุกคนที่ให้คะแนน (ไม่ใช่เฉพาะที่ชอบ)
- Calibration (เวอร์ชันนี้) = 1 − ½·$\sum$|p_hist − p_rec| คือ 1 − total variation distance ของการกระจายแนวหนังระหว่างประวัติกับ Top-K ⟶ 1 = สัดส่วนเหมือนกัน ; คืน `nan` ถ้าไม่มี genre
- genre คั่นด้วย `|` หรือ `,` ถูก split และนับซ้ำตามจำนวนเรื่อง

Part 2: loo_folds และ evaluate_config (1/2)

```python
def loo_folds(ratings_df, positive_threshold=4.0):
    folds = []
    for uid, g in ratings_df.groupby("user_id"):
        pos = g[g["rating"] >= positive_threshold]
        if pos.empty: continue
        hold = pos.sort_values("rating").iloc[-1]
        folds.append((str(uid), ratings_df.drop(index=hold.name),
                str(hold["item_id"])))
    return folds

def evaluate_config(X, profile_method, k=3, alpha=1.0, beta=1.0,
        folds=None,
                    candidate_mode="Full Catalog", n_neg=20, seed=42):
    folds = folds or loo_folds(ratings)
    rng = np.random.default_rng(seed)
    rows = []; global_recs = []
    for uid, train, heldout in folds:
        seen = set(train.loc[train["user_id"].astype(str) == uid,
                "item_id"].astype(str))
        all_c = [iid for iid in items["item_id"] if iid not in seen]
        if heldout not in all_c: all_c.append(heldout)
        if candidate_mode == "Sampled Negatives":
            neg = [x for x in all_c if x != heldout]
            sampled = list(rng.choice(neg, size=min(n_neg, len(neg)),
                    replace=False)) if neg else []
            candidates = [heldout] + sampled
        else:
```

อธิบายโค้ด

Evaluation engine ของทั้งสามเล่ม

- LOO fold = (uid, train ที่ไม่มี held-out, held-out = positive ที่ rating สูงสุด) ; ผู้ใช้ไม่มี positive ถูกข้าม
- candidate: Full Catalog = ทุกเรื่องที่ไม่อยู่ใน train ; Sampled Negatives = held-out + negative สุ่ม `n_neg` (seed คงที่) ⟶ ตัวเลขมักดูดีกว่า ห้ามเทียบข้าม protocol
- `if heldout not in all_c: all_c.append(heldout)` รับประกันว่าคำตอบอยู่ในกลุ่ม candidate เสมอ
- metric ต่อผู้ใช้ ⟶ `detail` ; `summary` = ค่าเฉลี่ย (nan ถูกข้าม) + Coverage/Gini จาก `global_recs`
- ชื่อคอลัมน์สร้างจาก f-string ตาม `k` (เช่น `HR@3`) ⟶ Part 4 ของ LAB 2 อ้างชื่อเหล่านี้
- input มี 3 มิติ: `X` (space), `profile_method` (+α, β), `folds/candidate_mode` (protocol) ⟶ LAB 2 วน grid บนสามมิตินี้

Part 2: loo_folds และ evaluate_config (2/2)

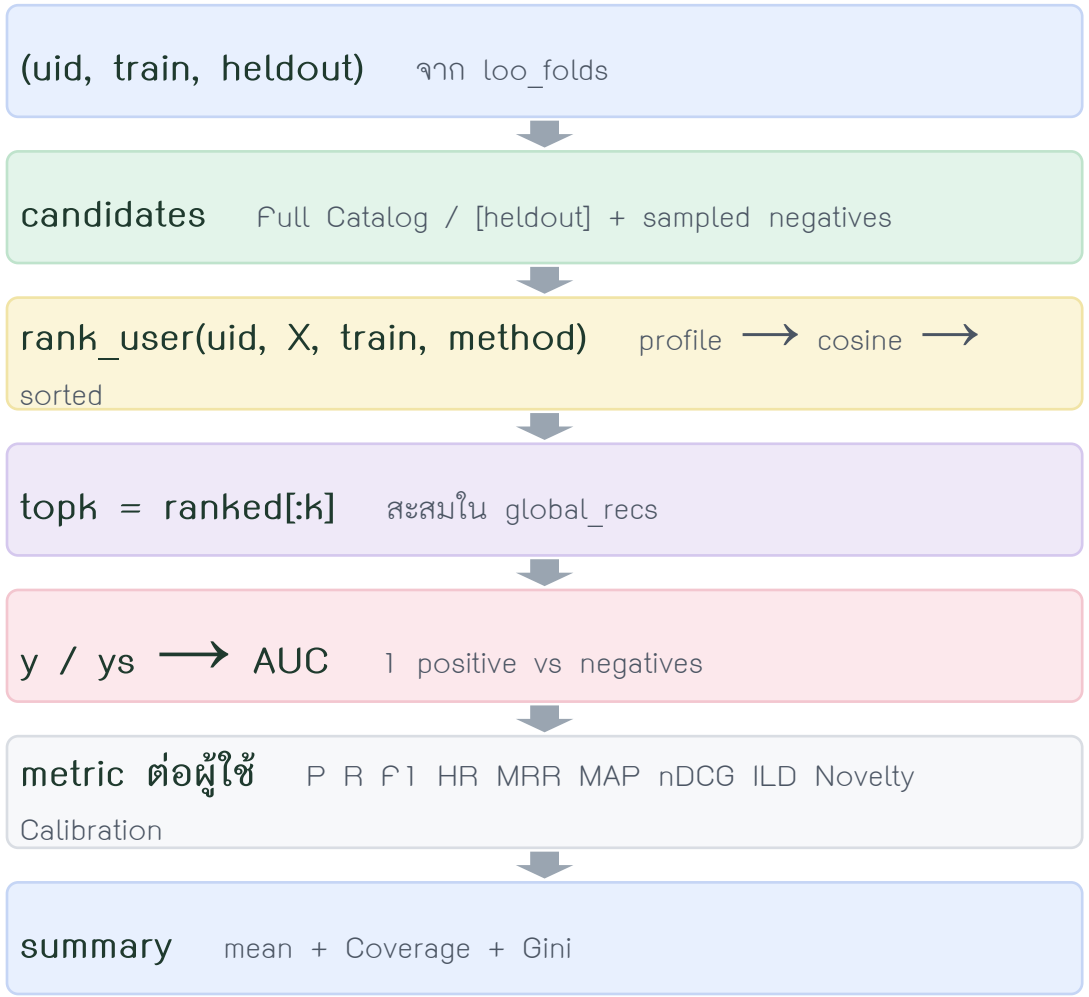
```
            candidates = all_c
        ranked, score, _ = rank_user(uid, X, train, profile_method,
                candidates=candidates, alpha=alpha, beta=beta)
        topk = ranked[:k]; global_recs += topk
        y = np.array([1 if iid == heldout else 0 for iid in ranked])
        ys = np.array([score[ITEM_INDEX[iid]] for iid in ranked])
        auc = roc_auc_score(y, ys) if len(np.unique(y)) == 2 else np.nan
        rows.append({"user_id": uid, "heldout": heldout,
                     f"Precision@{k}": precision_at_k(topk, [heldout],
                             k),
                     f"Recall@{k}": recall_at_k(topk, [heldout], k),
                     f"F1@{k}": f1_at_k(topk, [heldout], k),
                     f"HR@{k}": hr_at_k(topk, [heldout], k),
                     "MRR": reciprocal_rank(ranked, [heldout]),
                     f"MAP@{k}": ap_at_k(topk, [heldout], k),
                     f"nDCG@{k}": ndcg_at_k(topk, [heldout], k),
                     "AUC": auc, "ILD": ild(topk, X), "Novelty":
                             novelty(topk),
                     "Calibration": calibration_genre(uid, topk, train),
                     "TopK": " | ".join(topk)})
    detail = pd.DataFrame(rows)
    summary = detail.select_dtypes(include=np.number).mean().to_dict()
            if len(detail) else {}
    summary["Coverage"] = coverage(global_recs)
    summary["Gini"] = gini_coefficient(global_recs)
    return pd.Series(summary), detail
```

อธิบายโค้ด

Evaluation engine ของทั้งสามเล่ม

- LOO fold = (uid, train ที่ไม่มี held-out, held-out = positive ที่ rating สูงสุด) ; ผู้ใช้ไม่มี positive ถูกข้าม
- candidate: Full Catalog = ทุกเรื่องที่ไม่อยู่ใน train ; Sampled Negatives = held-out + negative สุ่ม `n_neg` (seed คงที่) ⟶ ตัวเลขมักดูดีกว่า ห้ามเทียบข้าม protocol
- `if heldout not in all_c: all_c.append(heldout)` รับประกันว่าคำตอบอยู่ในกลุ่ม candidate เสมอ
- metric ต่อผู้ใช้ ⟶ `detail` ; `summary` = ค่าเฉลี่ย (nan ถูกข้าม) + Coverage/Gini จาก `global_recs`
- ชื่อคอลัมน์สร้างจาก f-string ตาม `k` (เช่น `HR@3`) ⟶ Part 4 ของ LAB 2 อ้างชื่อเหล่านี้
- input มี 3 มิติ: `X` (space), `profile_method` (+α, β), `folds/candidate_mode` (protocol) ⟶ LAB 2 วน grid บนสามมิตินี้

ผังของ evaluate_config ต่อหนึ่ง fold



ทำไมเปรียบเทียบได้ยุติธรรม

- fold ชุดเดียวกัน (ส่ง `folds` เข้ามา) → held-out เดิมทุก configuration
- held-out ถูก drop จาก `train` ก่อนสร้างโปรไฟล์และไม่อยู่ใน `seen` → ไม่มี leakage แต่ยังเป็น candidate
- สิ่งเดียวที่ต่างระหว่างแถวของ `lab1_results` คือ `X` และ `profile_method`

4

Vector Spaces, 3 Profiles และ Offline Evaluation

Part 2 (ท้าย) – Part 4

- `VECTOR_SPACES` = 3 transformation ของ TF-IDF
- Part 3 เทียบ Top-3 และ cosine ระหว่างโปรไฟล์ของผู้ใช้ตัวอย่าง
- Part 4 ประเมิน 3 × 3 = 9 configuration ภายใต้ LOO + Full Catalog

Part 2-3: VECTOR_SPACES และการเทียบ 3 profile

```
VECTOR_SPACES = {m: transform_vectors(X_tfidf_raw, m)
                 for m in ["Raw+L2", "Center+L2", "Standard+L2"]}
for n, X in VECTOR_SPACES.items(): print(n, X.shape)

# Part 3 — User Profile 3 รูปแบบ
example_user = sorted(ratings["user_id"].unique())[0]
X = VECTOR_SPACES["Raw+L2"]
rows = []; profiles = {}
for method in PROFILE_METHODS:
    p = get_profile(example_user, X, ratings, method)
    profiles[method] = p
    ranked, score, _ = rank_user(example_user, X, ratings, method, k=3)
    rows.append({"Profile": method,
                 "Top-3": " | ".join(items.iloc[ITEM_INDEX[i]]["title"]
                          for i in ranked)})
display(pd.DataFrame(rows))

names = list(profiles)
M = np.array([[cosine_similarity(profiles[a].reshape(1, -1),
        profiles[b].reshape(1, -1))[0, 0]
               for b in names] for a in names])
display(pd.DataFrame(M, index=names, columns=names).round(4))
```

อธิบายโค้ด

ดูสองอย่าง: อันดับ และ ทิศทาง

- VECTOR_SPACES = dict ชื่อ ⟶ เมทริกซ์ ; GUI และ Part 4 หยิบตามชื่อ
- ตารางแรก: Top-3 ของผู้ใช้คนแรกต่อ profile ใน space Raw+L2 (ใช้ ratings เต็ม เพราะเป็นการสาธิต)
- เมทริกซ์ M 3 × 3 = cosine ระหว่างโปรไฟล์ ⟶ บอกว่าสูตรต่างกันให้ทิศทางต่างกันแค่ไหน แม้ Top-3 อาจเหมือนกัน (candidate มีแค่ ~9 เรื่อง)
- ค่านอกแนวทแยงต่ำ = โปรไฟล์ต่างกันจริง ; มักเกิดเมื่อผู้ใช้มีหนัง rating ≤ 2 หลายเรื่อง (Rating-centered / Rocchio ใช้ negative, Positive-only ไม่ใช้)

Part 4: Offline Evaluation ทุก Transformation × Profile

```
folds = loo_folds(ratings)
rows = []
for tname, X in VECTOR_SPACES.items():
    for prof in PROFILE_METHODS:
        s, _ = evaluate_config(X, prof, k=3, folds=folds,
                candidate_mode="Full Catalog")
        rows.append({"Transformation": tname, "Profile": prof,
                **s.to_dict()})
lab1_results = pd.DataFrame(rows)
display(lab1_results.round(4))
```

อธิบายโค้ด

9 configuration บน protocol เดียว

- `folds` คำนวณครั้งเดียวแล้วส่งเข้าไปทุกครั้ง ⟶ held-out เดิมทุกแถว
- `candidate_mode="Full Catalog"` และ `k=3` คงที่ ⟶ ต่างกันเฉพาะ `X` กับ `prof`
- ตาราง `lab1_results`: 9 แถว × (Precision@3, Recall@3, F1@3, HR@3, MRR, MAP@3, nDCG@3, AUC, ILD, Novelty, Calibration, Coverage, Gini)
- **วิธีอ่าน:** ดู HR/MRR/nDCG คู่กับ Coverage/Gini/ILD/Novelty/Calibration — profile ที่แม่นขึ้นอาจกระจุกขึ้น (Coverage ลด, Gini เพิ่ม)
- ILD ของ Center+L2 มักสูงกว่า Raw+L2 เพราะ cosine ใน space ที่ centering แล้วกระจายกว้างกว่า — ไม่ได้แปลว่าหลากหลายกว่าจริง

การอ่านตาราง lab1_results

กลุ่ม	คอลัมน์	ค่าที่ดี	สิ่งที่ควรระวัง
Accuracy / Ranking	Precision@3, Recall@3, F1@3, HR@3, MRR, MAP@3, nDCG@3, AUC	สูง	LOO มี relevant เดียว ⟶ Precision@3 ≤ 0.333 ; สุ่มได้ HR@3 ≈ 0.3 กับ ~9 candidate
Beyond — การกระจาย	Coverage (สูง), Gini (ต่ำ)	กระจายทั่ว catalog	profile ที่แม่นอาจแนะนำเรื่องเดิม ๆ ให้ทุกคน
Beyond — ความหลากหลาย/ใหม่	ILD, Novelty	ขึ้นกับเป้าหมาย	ILD ขึ้นกับ space ; Novelty ใช้ popularity จาก ratings เต็ม
Beyond — สอดคล้องกับผู้ใช้	Calibration	ใกล้ 1 = ตรงรสนิยมเดิม	สูงมากอาจเป็น filter bubble

ห้ามเลือก configuration จาก metric เดียว

5

Explain Profile และ GUI

Part 5 – Part 6: อธิบายโปรไฟล์ด้วยคำ TF-IDF และหน้าสาธิต Gradio

- `top_profile_terms(profile, n)` แปลงมิติที่มีน้ำหนักสูงสุดกลับเป็นคำ
- `gui_recommend` รวมทุกอย่าง: transformation, profile, α/β, K

Part 5: top_profile_terms

```python
def top_profile_terms(profile, n=12):
    terms = np.array(tfidf.get_feature_names_out())
    idx = np.argsort(profile)[::-1][:n]
    return pd.DataFrame({"term": terms[idx], "weight": profile[idx]})

for method, p in profiles.items():
    print(method); display(top_profile_terms(p, 10))
```

อธิบายโค้ด

โปรไฟล์ที่อ่านออกเป็นภาษาคน

- โปรไฟล์อยู่ใน space TF-IDF ⟶ มิติที่ i คือคำ `terms[i]` ; `argsort[::-1][:n]` เลือก n มิติที่ค่าสูงสุด
- เทียบสาม profile ของผู้ใช้เดียวกัน: Positive-only มีแต่คำจากหนังที่ชอบ ; Rating-centered / Rocchio อาจมีคำที่ค่า **ลบ** (จากหนังที่ไม่ชอบ) ซึ่งไม่โผล่ใน top-n แต่กดคะแนนหนังที่มีคำนั้น
- ใน space Center+L2 / Standard+L2 คำที่พบทุกเรื่องจะไม่ติดอันดับ เพราะถูกลบค่าเฉลี่ยออก
- ทำได้เฉพาะ TF-IDF — LAB 2 ที่ใช้ Word2Vec/BERT/SBERT จึงแสดง `profile_preview` เป็นค่าตัวเลข 20 มิติแรกแทน

Part 6: gui_recommend และ gr.Interface (1/2)

```python
import gradio as gr
def gui_recommend(user_id, transform_name, profile_method, alpha, beta,
        k):
    X = VECTOR_SPACES[transform_name]
    ranked, score, p = rank_user(str(user_id), X, ratings,
            profile_method, k=int(k),
                                 alpha=float(alpha), beta=float(beta))
    hist = ratings[ratings["user_id"].astype(str) ==
            str(user_id)].merge(
        items[["item_id", "title", "genre"]], on="item_id", how="left"
    )[["title", "genre", "rating"]].sort_values("rating",
            ascending=False)
    rec = []
    for rank, iid in enumerate(ranked, 1):
        r = items.iloc[ITEM_INDEX[iid]]
        rec.append({"Rank": rank, "Title": r["title"], "Genre":
                r["genre"],
                    "Score": round(float(score[ITEM_INDEX[iid]]), 4),
                           "Synopsis": r["synopsis"]})
```

อธิบายโค้ด

หนึ่งฟังก์ชัน สามตาราง

- `gr.Interface` แบบง่าย: input 6 ตัว ⟶ `gui_recommend` ⟶ output 3 DataFrame (ไม่ต้องผูก event เอง)
- ตัวเลือก Transformation/Profile มาจาก key ของ `VECTOR_SPACES` และ `PROFILE_METHODS` ⟶ เพิ่มแบบใหม่ใน dict แล้ว GUI เห็นทันที
- `rank_user` คืน `p` ⟶ แสดง Top Profile Terms = คำอธิบายว่าระบบเข้าใจผู้ใช้อย่างไร ; `score` ⟶ คอลัมน์ Score ของแต่ละคำแนะนำ
- α, β ส่งผลเฉพาะเมื่อเลือก Rocchio (ฟังก์ชันอื่นไม่รับ) ; ใช้ `ratings` เต็มเพราะเป็นการสาธิต ไม่ใช่ประเมิน
- แนวทางสอน: เลือกผู้ใช้ ⟶ ดู History ⟶ สลับ Profile ⟶ ดูว่า Top Terms และ Top-K เปลี่ยนอย่างไร ⟶ ลองปรับ β

Part 6: gui_recommend และ gr.Interface (2/2)

```
    return hist, top_profile_terms(p, 12), pd.DataFrame(rec)

demo = gr.Interface(
    fn=gui_recommend,
    inputs=[gr.Dropdown(sorted(ratings["user_id"].unique()),
            label="User"),
            gr.Dropdown(list(VECTOR_SPACES), value="Raw+L2",
                    label="Transformation"),
            gr.Dropdown(list(PROFILE_METHODS), value="Positive-only",
                    label="User Profile"),
            gr.Slider(0, 2, 1, 0.1, label="Rocchio alpha"),
            gr.Slider(0, 2, 1, 0.1, label="Rocchio beta"),
            gr.Slider(1, 5, 3, 1, label="Top-K")],
    outputs=[gr.Dataframe(label="User History"),
             gr.Dataframe(label="Top Profile Terms"),
             gr.Dataframe(label="Recommended Movies")],
    title="Chapter 3 – Content-Based Recommender")
demo.launch(debug=False)
```

อธิบายโค้ด

หนึ่งฟังก์ชัน สามตาราง

- `gr.Interface` แบบง่าย: input 6 ตัว ⟶ `gui_recommend` ⟶ output 3 DataFrame (ไม่ต้องผูก event เอง)
- ตัวเลือก Transformation/Profile มาจาก key ของ `VECTOR_SPACES` และ `PROFILE_METHODS` ⟶ เพิ่มแบบใหม่ใน dict แล้ว GUI เห็นทันที
- `rank_user` คืน `p` ⟶ แสดง Top Profile Terms = คำอธิบายว่าระบบเข้าใจผู้ใช้อย่างไร ; `score` ⟶ คอลัมน์ Score ของแต่ละคำแนะนำ
- α, β ส่งผลเฉพาะเมื่อเลือก Rocchio (ฟังก์ชันอื่นไม่รับ) ; ใช้ `ratings` เต็มเพราะเป็นการสาธิต ไม่ใช่ประเมิน
- แนวทางสอน: เลือกผู้ใช้ ⟶ ดู History ⟶ สลับ Profile ⟶ ดูว่า Top Terms และ Top-K เปลี่ยนอย่างไร ⟶ ลองปรับ β

สรุปแนวคิดของ Source Code LAB 1

Take-away

- **Part 0 + Core Functions ใช้ร่วมกันสามเล่ม:** `items/ratings/ITEM_INDEX/POPULARITY` และฟังก์ชัน transform / profile / rank / metric / folds / evaluate_config — LAB 2–3 เพียงเรียกด้วยเงื่อนไขต่างกัน
- **Transformation เปลี่ยน geometry:** Raw+L2 (ทิศทาง), Center+L2 (ต่างจากค่าเฉลี่ย, มีค่าลบ), Standard+L2 (ทุกมิติเท่ากัน) ⟶ โปรไฟล์และอันดับเปลี่ยนตาม
- **3 User Profile = 3 วิธีใช้ rating:** Positive-only ใช้เฉพาะชอบ ; Rating-centered ถ่วงด้วย rating − ค่าเฉลี่ยของผู้ใช้ (แก้ bias) ; Rocchio = α·liked − β·disliked (ปรับได้)
- **`get_profile` / `rank_user` เป็นทางเข้าเดียว** ที่รับ `method`, α, β, `candidates` ⟶ GUI และ evaluation ใช้โค้ดชุดเดียวกัน
- **`evaluate_config` = engine:** LOO fold คงที่, candidate protocol เลือกได้, 13 metric ทั้ง Accuracy และ Beyond-Accuracy ในตารางเดียว
- **อ่านผลคู่กันเสมอ** (HR/nDCG กับ Coverage/Gini/ILD/Novelty/Calibration) และ `top_profile_terms` ช่วยอธิบายว่าทำไมจึงแนะนำ

LAB 1 = พื้นฐาน • LAB 2 = ขยายเป็น experiment matrix • LAB 3 = ต่อยอดเป็น predictive pipeline